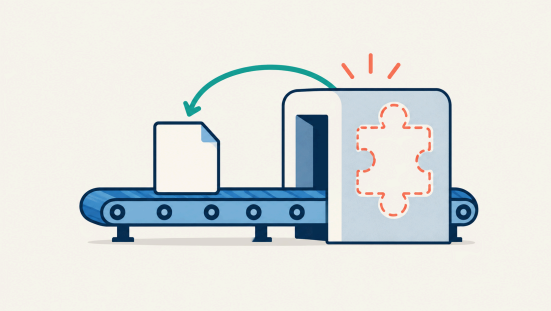
The Missing Test Suite

Where the bottleneck goes when code is cheap

Daniel Fridljand · danielfridljand.de



- About 35 minutes, then questions.
- A project I took from discovery to a live shadow deployment: an LLM automation for first-level customer support.
- Not a talk about prompts or models. A talk about where the time went, and about the part that ran like a software factory: a coding agent and CI turning each decision into a branch, an eval case and a green build overnight.

Early in the project, we had to decide: workflow or agent?

Worth a talk of its own: Workflow or Agent.
Today we take it as fixed: we built a workflow.

- Early in the project, we had to decide how much of the work the model does and how much code does: one agent holding every tool, or a workflow where code decides the order of operations.
- That decision is worth a separate presentation, and there is a post on it.
- In this talk we take it as fixed that we decided on the workflow. Everything that follows, the loop and the overnight fixes, runs on top of that workflow.

Code is cheap now.
So what becomes the bottleneck?

On a support automation I built: the experts who could tell whether a reply was right.

- The question hanging over coding agents: once code is cheap, what is slow?
- My answer, from one project: the people who know the business. Not because they are slow, but because they are the only check the system has.
- The rest of the talk: why that is, the loop we built around it, and what I would do differently.

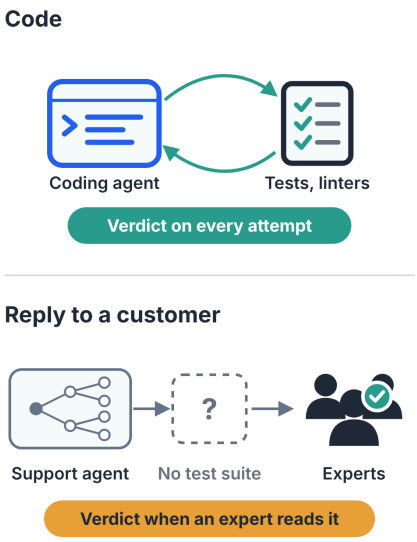
Why coding agents work

1. **Software is familiar.** Syntax, idioms and libraries are patterns the model has seen countless times in training.
2. **Every attempt gets a verdict.** Tests, linters and error messages tell the agent whether it worked, so the agentic loop runs without a human in it.

- Two reasons, and the second is the one that matters for this talk.
- The agent tries, the test suite answers, the agent tries again. Nobody has to read the code for the loop to make progress.
- Ask the room: how many of you let an agent run against a test suite unattended? That is the free verdict.

A reply to a customer has no test suite

- Code: the verdict comes **on every attempt**, for free.
- A reply: the verdict comes **when an expert reads it**.
- The model hasn't seen your escalation rules or your customers' records either.



- Neither condition holds for a business process.
- Records: a read tool, an endpoint the system can call, solves that.
- Rules: a read tool would cover them too, if anyone had written them down. Usually nobody has; they live in the heads of the people who handle the tickets.
- So the only check on a reply is a person who knows the business reading it. That is the missing test suite, and it is where the effort belongs.

The running example

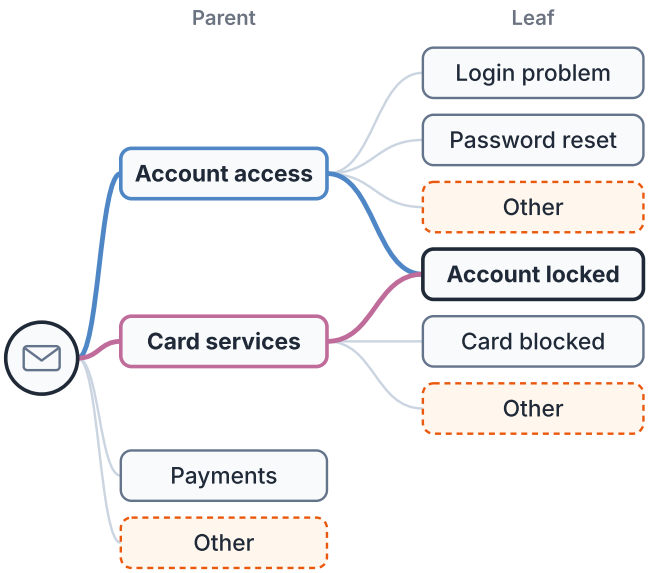
Customer · 2026-03-12 09:41

Hi, I'm Alex Carter and I bank with your Riverton branch. My account number is NSB01234 and I can't get in any more; it keeps saying the account is locked. Can you help me get back in?

Invented to protect the client; the project behind it is real.

- Say it out loud: the bank, Alex and the account are invented. The project is a real customer support automation.
- First-level support: the messages that come in before anyone specialised looks at them.
- A locked account is the commonest kind of request, and simple on the surface. Keep Alex in mind; we follow this ticket through the whole loop.

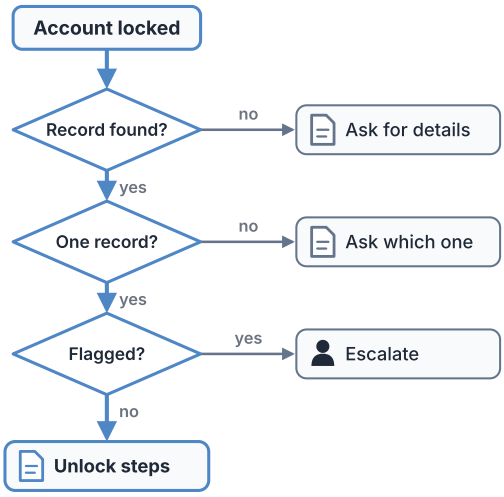
The workflow, step one: which kind of request is it?



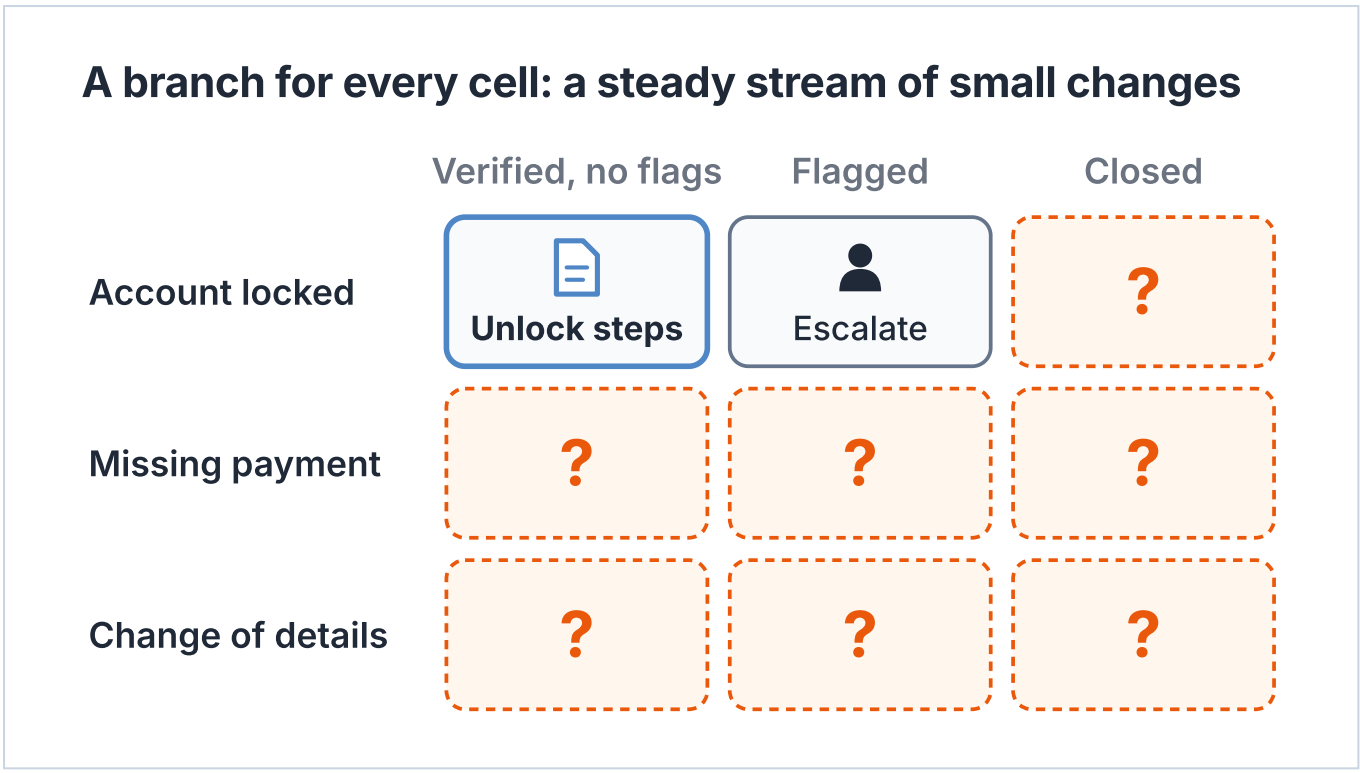
- The workflow's structure, briefly, because it is what the coding agent changes every night. More in Architecture of a First-Level Support Automation.
- A rooted DAG of intents, mostly two levels deep. One narrow LLM call picks the parent, a second picks the child.
- Every parent has an explicit "Other": coverage gaps land somewhere visible.
- Alex goes Account access → Account locked. A card-PIN lockout reaches the same leaf through Card services; one leaf, one plan.

Then code decides what happens

- The model **reads** the message: the intent and the account number.
- Code **decides** the order: look up, check the flag, then unlock or escalate.
- The security check comes first, every time, and the reply is a fixed template.

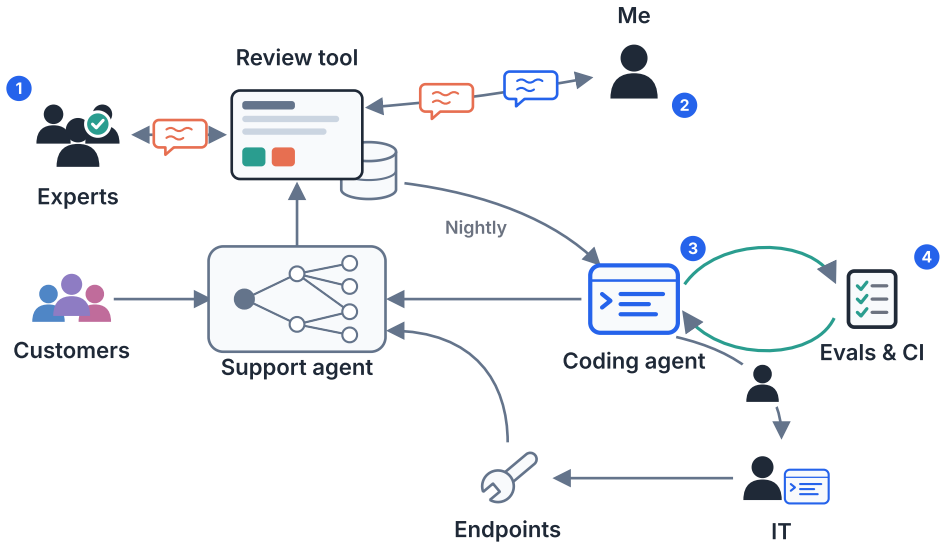


- Each leaf owns a small decision tree in code over the fetched data; each branch ends in a template and tool calls.
- This is the workflow we decided on. Why, and what I'd weigh differently today, is the other talk.
- What matters here: the business logic lives in code, in small, separate places. That is what makes a change cheap to specify and safe to hand to a coding agent.



- A workflow needs a branch for every cell: every kind of request against every state its record can be in.
- This toy grid already has seven open cells. We had many more than three of each.
- Each question mark is a decision someone has to make, and then a small change in code. That is the factory's work: many small, similar changes.
- Nobody can fill this grid from a desk. The people who answer the tickets know the answers, and they can't list them up front. That is what the loop is for.

The whole system



- The map for the rest of the talk; each section highlights its part.
- Customers write in, the support agent drafts a reply, the draft lands in a review tool.
- The numbers are the loop: 1 experts grade, 2 I specify the fix, 3 a coding agent implements it overnight, 4 evals and CI check it.
- Bottom right: endpoints, built by the client's IT. We'll get there.

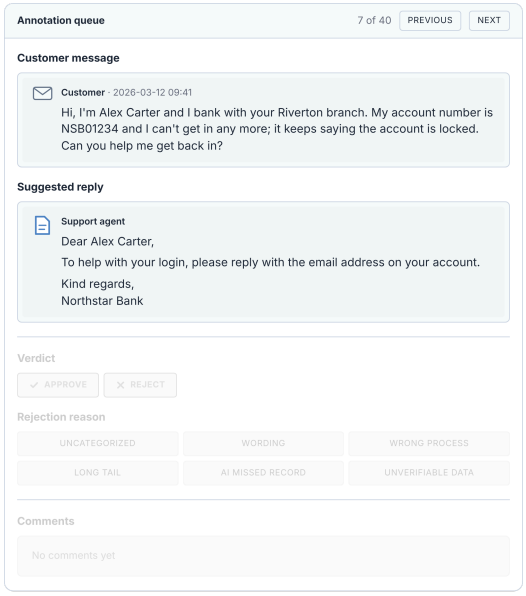
Shadow deployment: a draft for every real ticket, none sent



- The system drafted a reply to every real ticket. Nothing was sent automatically.
- The experts were the support staff who answered those tickets anyway. When one picked up a ticket, they graded the draft for it.
- Happy with it: paste it into the reply. Not happy: write their own.
- A draft reached a customer only when an expert approved it and sent it themselves.

Step 1: experts grade

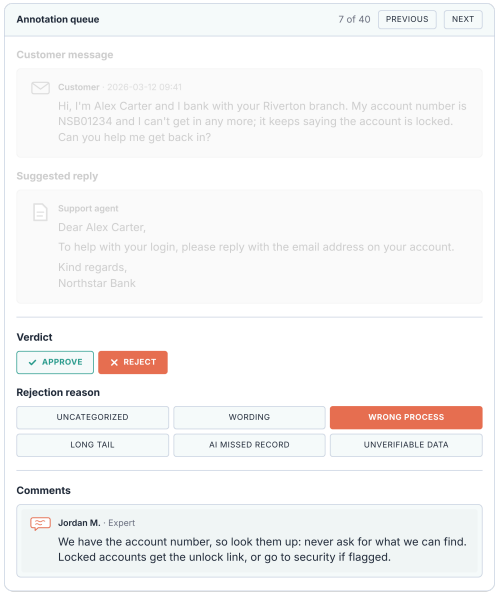
- The customer’s message and the proposed reply, nothing else.
- Graded when the expert picks up the ticket anyway.
- An approved draft is a reply they don’t have to write.



- A mockup, with the invented example.
- Before the tree had an Account locked intent, Alex’s message landed in “Other”, and the fallback agent wrote this: it asks Alex for the email address on the account.
- Ask the room: what’s wrong with it? (We already have the account number.)

One rejection, two rules nobody had written down

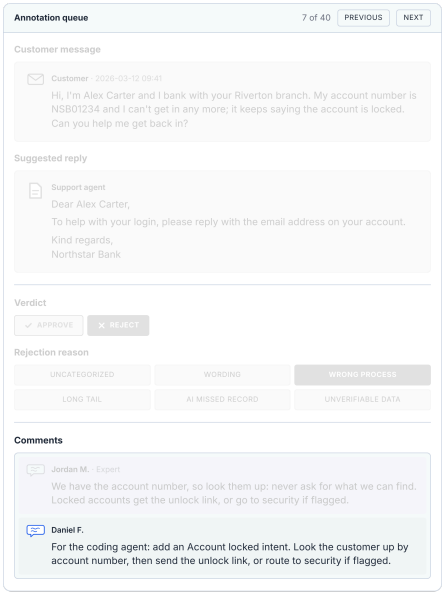
- **Never ask for what we can find.**
- **Locked accounts get the unlock link, or go to security if flagged.**
- Grading is the check a coding agent gets from its tests, and how the rules get written down.



- Verdict: reject. Reason: wrong process. And a comment.
- The people who know a business can rarely write its rules down in advance. Shown a draft, they see at a glance what is wrong with it.
- One bad reply brought out two rules. That is the best ratio you will get: you can’t interview your way to these.
- So grading does two jobs at once.

Step 2: I specify the fix

- In one sitting, for a batch of feedback.
- A plan for the coding agent next to the expert's comment: where the fix goes, what it does.
- Follow-up questions go in the tool, next to the ticket.



- I go through a batch of stored feedback in one sitting and add my comment next to each expert's: a high-level implementation plan for the coding agent.
- Where in the system the fix goes, and what it should do. Not the code.
- This comment is the factory's work order.

Designed so a fix fits in two sentences

Most feedback becomes a change **in one place**:

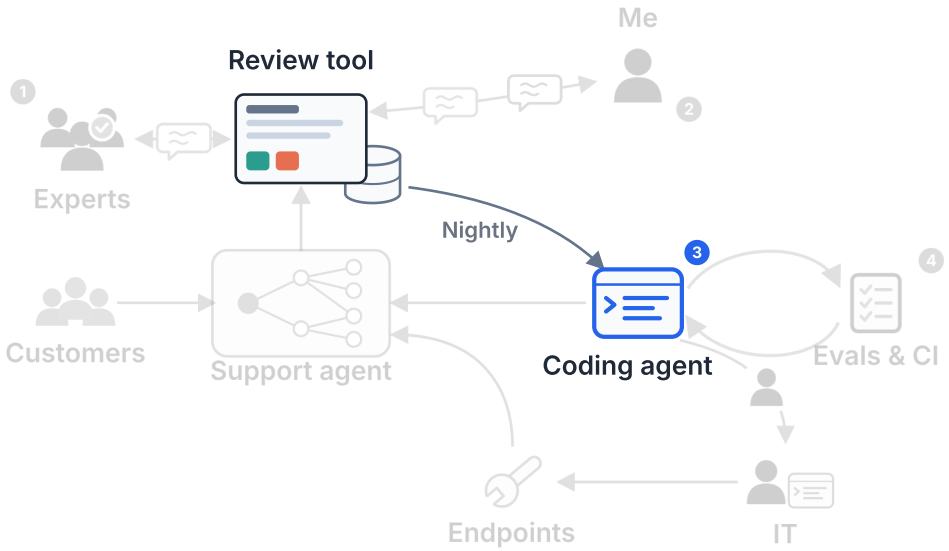
- a new intent, a kind of request to recognise,
- a new branch under an intent it already knows,
- a second parent for an existing intent.

A change like that I can describe in two sentences and trust a coding agent to build.

The design work came first: which decisions code makes and which the model makes.

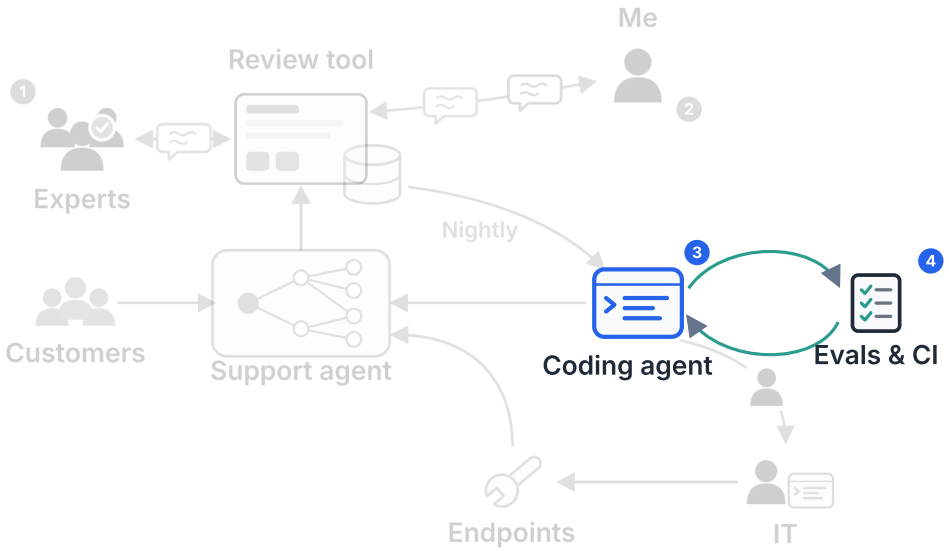
- My job had two parts. The first was the system's structure: requirements, and the split of decisions between code and model. It took a few rounds, but it is mostly work done up front.
- The point of that structure, for the factory: most expert feedback lands in one place. A new leaf in the DAG, a new branch in a leaf's tree.
- A small, local change is one I can specify briefly and check easily. That is what makes it safe to hand to an agent unattended.
- If every fix had cut across the system, no two-sentence plan would have been enough, and the nightly run wouldn't work.

Step 3: the coding agent implements it overnight



- A nightly run picks up every ticket where I have specified a fix, all at once.
- Usually that means adding a new intent to the intent DAG, or a new branch to the rules behind one. For Alex's ticket: a new Account locked intent.
- With a coding agent, a fix I had specified was a branch, an eval case and a green build by the next morning, with little of my time.

Step 4: evals and CI check it



- The agent adds the case to the eval set: here Alex's message and the reply it should get.
- CI re-runs the whole suite, so earlier fixes stay fixed.
- This step is an ordinary test suite, and it is the easy part: once a case is written down, the rest is ordinary engineering.
- Notice what happened: the expert's verdict became a test. The missing test suite gets written one rejected draft at a time.

Evals: one trace per case, tied to a commit

- Eval cases live in git as JSONL, synced to Langfuse as datasets.
- Each run: one trace per case, with input, output and scores.
- Every trace carries its commit, branch and environment.

Separate from unit tests

Unit tests say whether the code works. Evals say how well the system answers, over time, run against run.

- How we ran step 4, in more detail in Langfuse in Production.
- Evaluators return structured scores, such as intent match and correct tools selected, plus run-level aggregates.
- We never throw a run away: open any past run, drill into a failed case, and see which step failed, what the model saw and returned, and what the scorer said.
- That full trace, tied to a commit, is what lets an agent work on a failure without me.

When an eval fails, agents triage it

1. **Discovery** lists the failing traces.
2. **Analyzer**, one trace at a time: finds the root cause, fixes it or leaves a note on the trace.
3. **Rerunner** re-runs the failed cases.

Root causes, each with its own fix

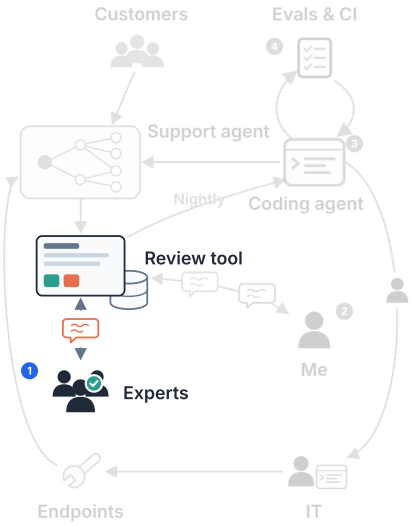
- Eval data wrong
- Mock doesn’t match the API
- Prompt wording
- Code bug
- Flaky: re-run, note it

Never: editing a real customer message to make its test pass.

- One command runs the triage loop with three specialised subagents, passing typed payloads between them.
- The analyzer runs one trace at a time because each may modify source files; parallel edits would conflict.
- It classifies each failure into one of seven categories (I’ve merged the mock ones), each with its own remediation guide.
- The hard rule: raw customer-interaction fixtures are off limits. Editing one makes a test pass without making the system better. That boundary is in the agent’s system prompt, not left to convention.
- And fix eval data before prompts: otherwise you “improve” a metric by tightening a prompt when the real fix was a wrong label.

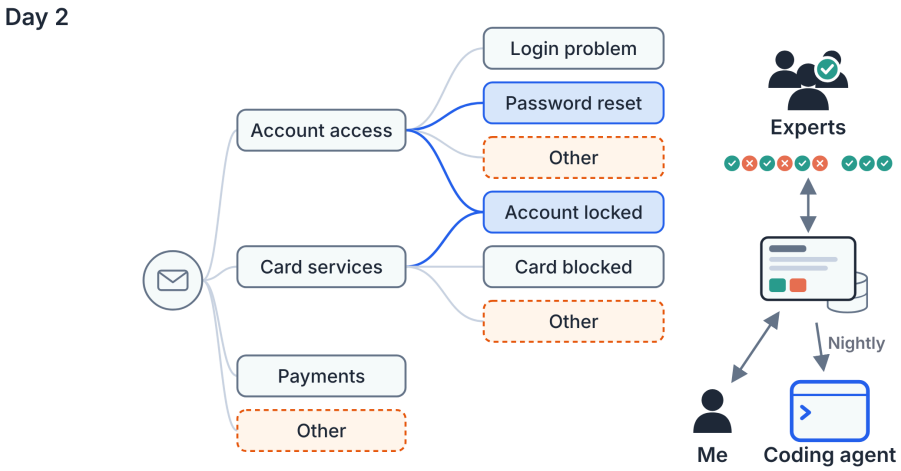
The factory checks against my plan; only an expert checks against the business

- The eval case is written from **my** plan.
- A green build says the code does what I asked, not that I asked for the right thing.
- So I resubmit the rejected ticket, and an expert checks the new draft.



- Then the fix ships, and I resubmit the rejected ticket from the review tool: the support agent drafts a new reply, and the draft goes back into the shared queue.
- Whichever expert picks it up checks the fix on the ticket that exposed the error. The eval case can’t do that.
- A rejected ticket leaves the loop only when an expert approves it.
- This is the limit of the software factory here: everything it verifies, it verifies against a spec. For code, the spec is the tests. For a business process, the spec is in the experts’ heads.

After one round: the intent DAG grew overnight



- Day 1: several messages at once, each runs through the DAG, experts grade as they answer. Most grades are checks.
- The crosses' comments pile up next to their tickets. I take the pile in one sitting; for one, I first ask a question, and the expert answers when they get to it.
- At night the coding agent takes all my plans at once: two new leaves, and a second parent for Account locked.
- Day 2, the state on the slide: the resubmitted tickets reach their new leaves and come back approved. A rejected ticket leaves the loop only when an expert approves it.
- The post has this as an animation, from Day 1 on.

One developer, ten experts, no direct messages

A direct message

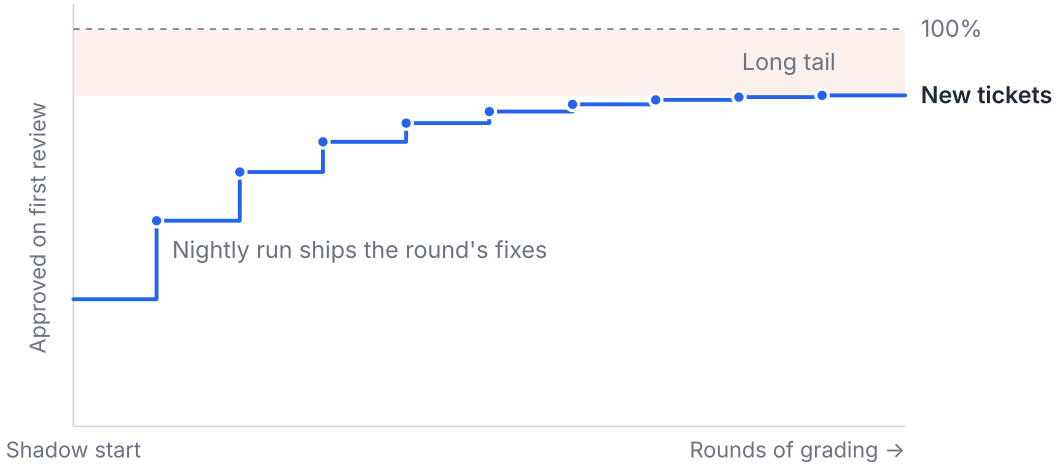
- is about one ticket,
- gets lost a week later,
- needs an answer now.

A comment in the review tool

- stays with its ticket,
- is there when I batch the next fixes,
- waits until someone is ready.

- Nobody messaged anybody. The queue was shared; each expert graded as they worked through their tickets.
- I went through the comments in one sitting, asked follow-ups next to the ticket, and the experts answered when they got to it.
- That asynchrony is what made ten experts workable for one developer.

What the loop is built to raise



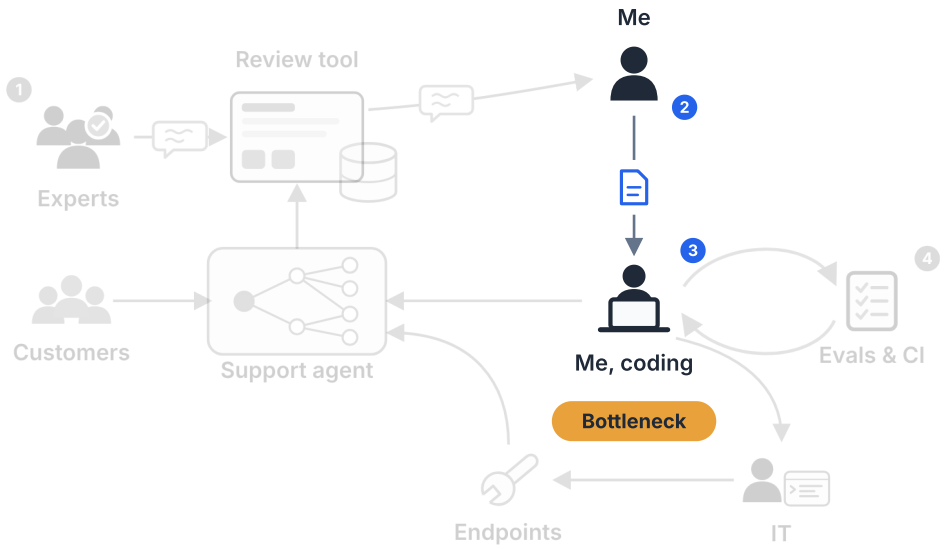
- Schematic, not measured.
- The share of drafts for new tickets the experts approve on first review. Resubmitted tickets don't count; they end up approved by construction.
- Each nightly run fixes the request types its round exposed, common ones first, so each step is smaller than the last.
- It levels off below 100%: rare and new kinds of request keep coming.
- A caution: an approval rate is not a cost reduction. Unless something flags which drafts are wrong, someone still reads all of them, and every ticket costs a read instead of a write. Good number for steering, poor number for a business case: An Approval Rate Is Not a Cost Reduction.

Every system has a bottleneck.
Fixing it doesn’t remove it; it moves it.

Eliyahu Goldratt, theory of constraints

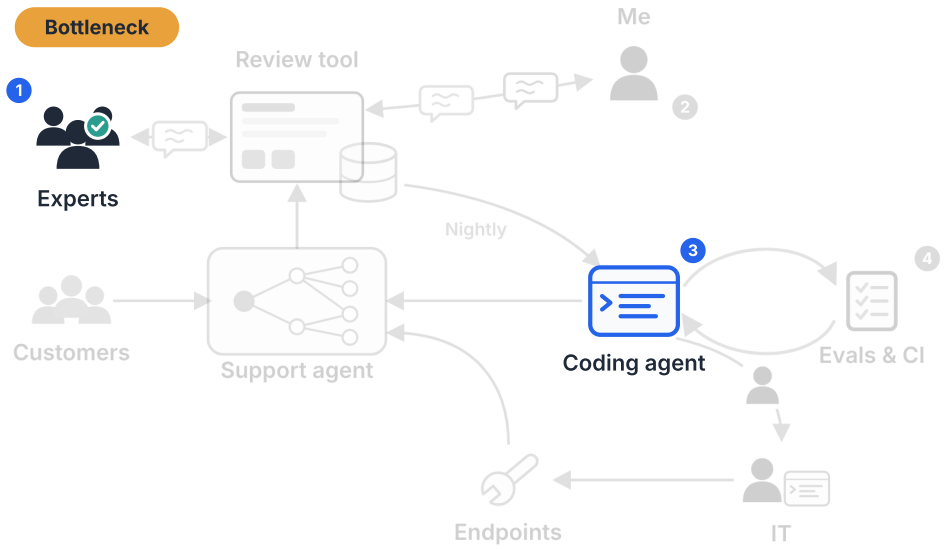
- Back to the opening question, with a frame for it.
- The theory of constraints starts from one observation: speed up anything that isn’t the bottleneck and nothing gets faster.
- So: where was ours, before and after coding agents?

Without coding agents: every fix goes through me



- Without a coding agent, I read the feedback, decide the fix, and write the code. Writing the code is the slow part.
- The experts would have waited for me.

With coding agents: the experts' time sets the pace



- With one, a fix I had specified was a branch, an eval case and a green build by the next morning, with little of my time.
- What limited progress was step 1: getting domain knowledge out of the experts' heads.
- That is what the missing test suite costs: we learned whether a reply was right only when an expert had time to read it.

Reviewers' time is a hard input

It cost them less than it might have

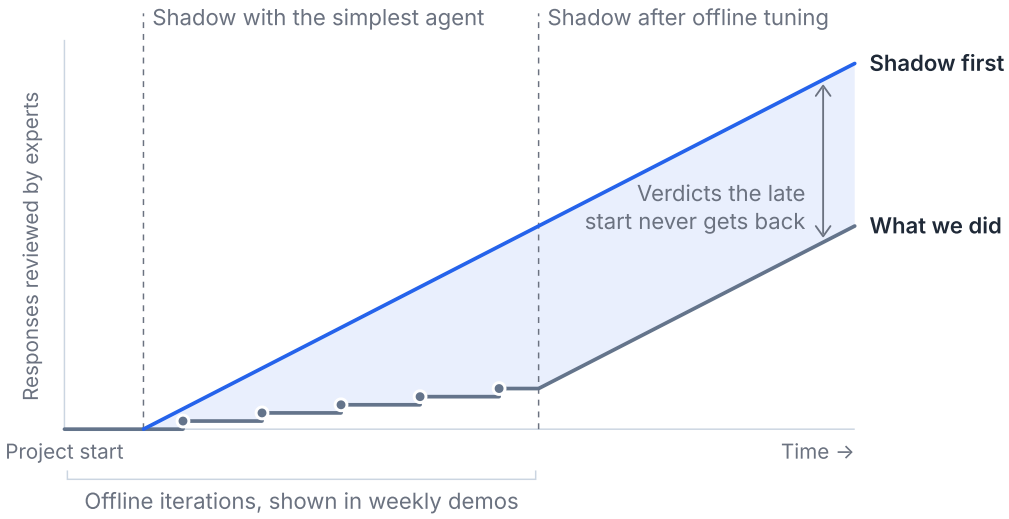
- They answered each ticket anyway.
- An approved draft was a reply they didn’t write.

It still cost them

- One more item in a full day.
- They may suspect they are training their own replacement.
- Engagement and sponsorship can’t be counted on.

- We were fortunate on both counts: the reviewers engaged with it, and the work had visible sponsorship from the client’s leadership, which kept both the quantity and the quality of the feedback up.
- When you estimate a project of this shape, the reviewers’ availability is a hard input, not a detail to sort out later.
- Two things follow, and they are the next two sections: get them grading as early as possible, and make each grade quick to give.

Get into shadow deployment sooner



- Schematic, not measured: the same reviewer capacity in both paths.
- We went live in shadow only once the offline numbers on historic tickets looked acceptable. That has it backwards.
- Before shadow, the experts saw only the handful of responses I demoed each week. The grades they could have given in those weeks are lost for good; the gap never closes.

Replaying old tickets is the weaker signal

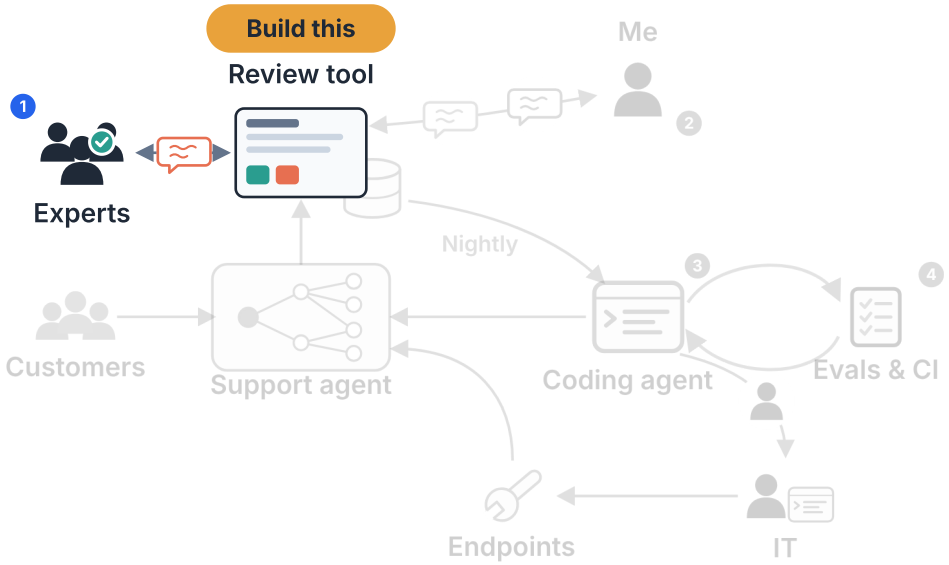
- A replayed ticket runs against **today's** records, not the customer's.
- A month later, Alex's account may already be unlocked.
- Only shadow deployment runs against live records.

Replay is good for: checking that fixes stay fixed.

Replay is poor for: finding out what a good reply looks like.

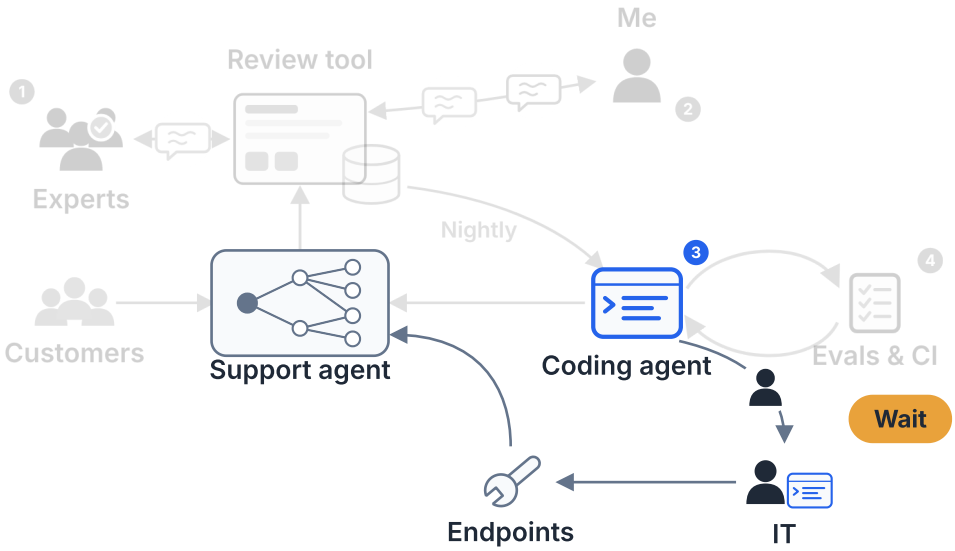
- The archive can't tell you whose records have changed since.
- So the offline numbers we waited for were not only late, they were the worse measurement.
- Keep replay as a regression suite, once you know what a good reply looks like.

Build the review screen yourself, not the storage behind it



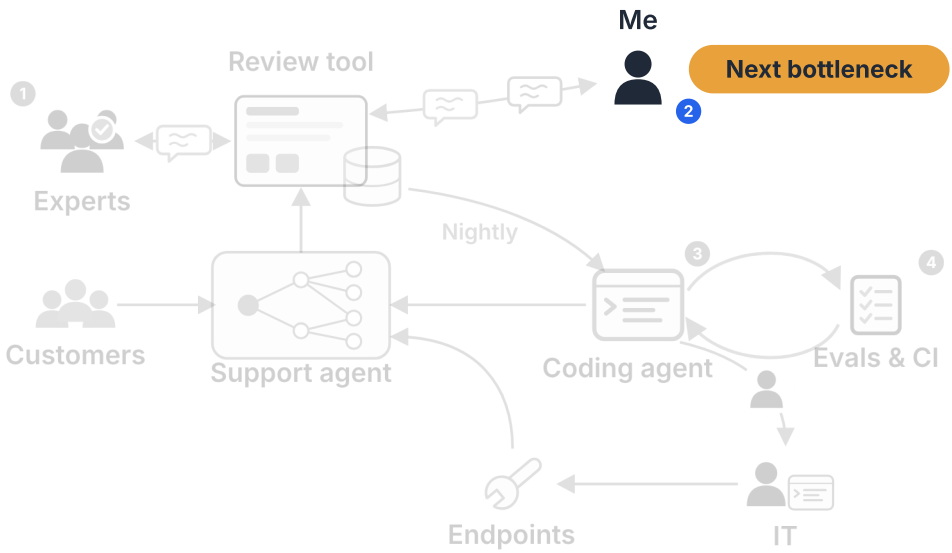
- With a coding agent it is tempting to build all of it. I built ours that way, store included; I wouldn't again.
- The screen is worth writing: the surface the experts touch every day, their only channel to you. Show the ticket and the reply and nothing else; every click you remove gives reviewers time back.
- Queues, assignment, score storage, history: quick to write, slow to get right. Build the screen on an existing tool, such as Langfuse's annotation queues.
- Same reason SaaS isn't dead however cheap code gets: Jamin Ball's argument that cheaper building leaves build-or-buy intact.

The other wait: endpoints



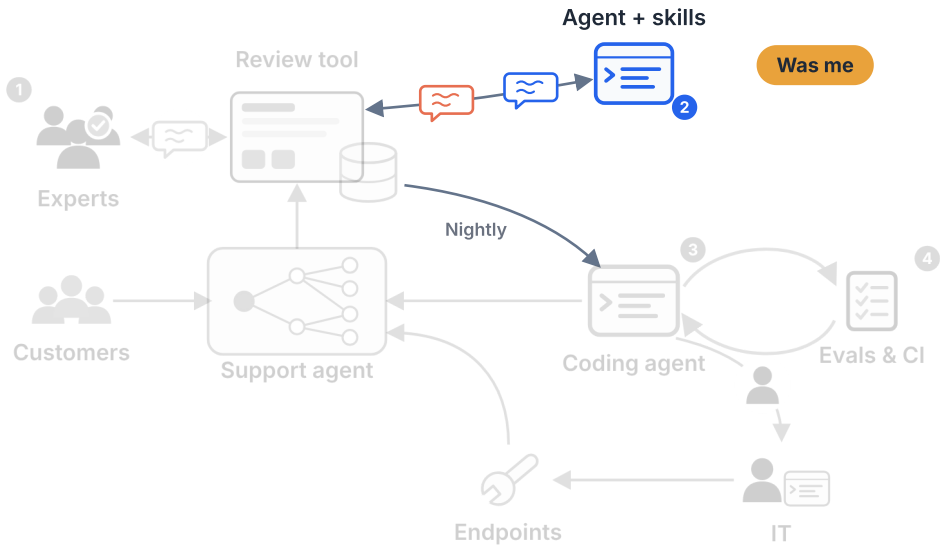
- A reply is only as good as what the system can look up, so every new kind of answer needed a new endpoint, such as one for account status. Those belonged to the client's IT.
- IT worked API-first: I committed an OpenAPI spec, and it was usually implemented within a day.
- The one handoff outside the review tool, and it showed: I pinged someone in chat, who pointed their own coding agent at the spec. Two coding agents, with a person on each end passing messages.
- I don't know the right fix. Maybe as plain as an MCP server through which my agent files the request.

The next bottleneck: who translates the feedback



- Step 2 was me. That puts one engineer on the critical path of every improvement; once the reviewers grade steadily, it is the next bottleneck.
- Handing step 2 to the coding agent on the raw feedback gave worse results than doing it myself. Skill files and targeted guidelines narrowed the gap without closing it.
- More specific skills, or a subagent dedicated to the translation, might close the rest.

Could the agent write its own skills?



- The more interesting version changes who writes the skills. An agent with persistent memory, something like Hermes, could rewrite them from each batch of grades. The experts would be the only people left in the loop.
- A skill is only a prompt, not a guarantee.
- The risk is drift: the skills slowly stop matching how the business works. The only thing that would catch it is the experts' grading, which is already the bottleneck.
- I haven't solved this. I think it is solvable, with a better loop rather than more of my time.

Start at the bottleneck

1. **Settle what the loop should improve**, and what that would save.
2. **Build the review screen and shadow deployment first**, with the simplest system that can draft a reply behind it.
3. **Then grow the rules from the grades**, not from a whiteboard.

- Coding agents didn’t remove the hard part of this project; they made it obvious where it was.
- If I started a project of this shape again, this is the order.
- Lead into the closing line.

Until someone who knows the business reads the output every day, every design decision is a guess.

danielfridljand.de/post/the-missing-test-suite

- Thank you. Questions.
- Likely questions: How did you get the experts to engage? What approval rate did you reach? (Don’t quote numbers the post doesn’t.) Why not let the agent handle everything from day one? How do you handle personal data in the review tool?